

DistilBERT trên FPGA — đánh giá AXI4_A/B_READER + AXI4_C_WRITER (Hướng B)

Notebook này thay thế `CaiTien-Tue.ipynb` (bản cũ, viết cho kiến trúc Giai đoạn 0 — dùng thanh ghi `A_1` / `A_2` tách đôi con trở 64-bit và **không transpose A**). Kiến trúc hiện tại (Hướng B, xem `01_design/docs/task.md`) đã đổi:

- `A_addr` / `B_addr` / `C_addr` : mỗi cái chỉ **1 thanh ghi 32-bit** (địa chỉ vật lý thuần), không còn tách `_1` / `_2` như bản cũ.
- **A bắt buộc phải transpose trước khi ghi buffer**: `AXI4_A_READER` đọc DDR theo layout `A_col[k][n]` (1 burst = tối đa 64 phần tử cùng `k`), khác hẳn layout `A[n][k]` thông thường. Quên bước này sẽ cho kết quả "trông hợp lý nhưng sai" — không crash, không báo lỗi, chỉ sai số — đây chính là lỗi mà bản `CaiTien-Tue.ipynb` cũ mắc phải ở toàn bộ phần tích hợp DistilBERT (`FPGAQuantizedLinear` , `FPGAQKVAttention` ...).
- **B không cần transpose** (giữ nguyên layout `B[k][m]` gốc).

Giới hạn phần cứng cần nhớ khi chọn layer để offload

- **N (số hàng A/C, thường là số token) ≤ 64** — độ sâu bank A theo hàng cố định 64.
- **K (chiều chung A/B) ≤ 768** — bank A tổng hợp cứng độ sâu 768 (`DUAL_PORT_BRAM_8B` , `MAX_K` trong `mmult_accel.h`). Với DistilBERT (`hidden_size=768`):
 - Q/K/V projection (`Linear(768,768)` , `K=768`): **offload được**, đúng khít giới hạn.
 - FFN up-projection (`Linear(768,3072)` , `K=768`): **offload được** (không dùng trong notebook này, để dành hướng mở rộng sau).
 - FFN down-projection (`Linear(3072,768)` , `K=3072`): **KHÔNG offload được** trong 1 lần gọi (vượt giới hạn K) — vẫn chạy CPU. Notebook này chỉ offload Q/K/V, đúng scope bài báo gốc.
- **M (số cột B/C) không giới hạn cứng** — được xử lý bằng blocking nội bộ (`BLOCK_M=256`).

Chuẩn bị trước khi chạy trên board thật

1. Copy bitstream mới nhất (`.bit` + `.hwh` cùng tên) lên KV260, cùng thư mục với notebook này. Sửa biến `BITSTREAM_PATH` ở Cell tiếp theo cho khớp tên file thật bạn vừa Export Hardware (sau khi build xong bản `AXI4_C_WRITER` gộp-burst — xem `01_design/docs/task.md`).

2. Mở chính notebook này từ **Jupyter server của KV260** (`http://<IP-board>:9090`), không phải máy cá nhân — vì `import pynq` chỉ có trên PYNQ image.
3. Cần cài sẵn `transformers` / `torch` trên board (thường đã có sẵn trong image PYNQ dùng cho đề án này, xem các notebook cũ đã chạy `AutoModel.from_pretrained` được).

Mỗi cell chạy độc lập bằng `Shift+Enter` ; biến ở cell trước tồn tại cho cell sau (cùng 1 kernel). Nếu treo/lỗi giữa chừng, dùng `Kernel → Restart` rồi chạy lại từ đầu.

Phần 1 — Nạp Overlay + hàm `call_fpga()` dùng chung

`call_fpga()` là **nguồn sự thật duy nhất** cho cách gọi phần cứng — mọi class phía dưới (`FPGAQuantizedLinear` , `FPGAQKVAttention`) đều gọi lại đúng hàm này, không tự viết lại logic transpose/thanh ghi riêng (khác bản `CaiTien-Tue.ipynb` cũ, nơi mỗi class tự chép lại 1 bản logic gọi phần cứng — dễ quên transpose ở 1 chỗ mà quên sửa hết).

```
In [1]: from pynq import Overlay, allocate
import numpy as np

# [SỬA TRƯỚC KHI CHẠY] Đổi tên file .bit khớp bản build mới nhất bạn vừa Export Har
BITSTREAM_PATH = "matmul6.bit"

overlay = Overlay(BITSTREAM_PATH)
accel_ip = overlay.mmult_accel_0

# In ra register map thật để xác nhận tên field trước khi dùng – không hard-code of
# vì mỗi lần re-package IP/Synthesis lại có thể đổi thứ tự thanh ghi.
print(accel_ip.register_map)
```

```
RegisterMap {
  CTRL = Register(AP_START=0, AP_DONE=0, AP_IDLE=1, AP_READY=0, RESERVED_1=0, AUTO_R
ESTART=0, RESERVED_2=0, INTERRUPT=0, RESERVED_3=0),
  GIER = Register(Enable=0, RESERVED=0),
  IP_IER = Register(CHAN0_INT_EN=0, CHAN1_INT_EN=0, RESERVED_0=0),
  IP_ISR = Register(CHAN0_INT_ST=0, CHAN1_INT_ST=0, RESERVED_0=0),
  A_addr = Register(A_addr=write-only),
  B_addr = Register(B_addr=write-only),
  C_addr = Register(C_addr=write-only),
  N = Register(N=write-only),
  K = Register(K=write-only),
  M = Register(M=write-only),
  update_A = Register(update_A=write-only)
}
```

```
In [2]: def call_fpga(A_np, B_np, accel_ip, update_A=1, A_buf=None, B_buf=None, C_buf=None)
        """
        Chạy (N,K) x (K,M) => (N,M) tren FPGA, dung dung register map A_addr/B_addr/C_a
        (kien truc Huong B) + transpose A truoc khi ghi buffer (bat buoc, xem markdown
```

A_np: ma tran hoat hoa/dau vao, shape (N, K), dtype int8 – layout THUONG [n][k]
 B_np: ma tran trong so, shape (K, M), dtype int8 – layout thuong [k][m], KHONG
 update_A: 1 = nap lai A vao BRAM (dung khi A doi – vd lan goi Q dau tien trong
 0 = dung lai A da nap tu lan goi truoc (dung cho K/V, cung input voi
 A_buf/B_buf/C_buf: neu truyen san (buffer PYNQ da allocate tu truoc, vd de tai
 nhiều lần gọi liên tiếp tránh alloc/free liên tục) – ham se GHI DE NO
 KHONG tu allocate lai. Neu None, ham tu allocate moi (tien loi cho 1

Tra ve: (C_np, C_buf) – C_np la numpy array da copy ra khoi buffer DMA, C_buf l
 (tra lai de goi tien len bo nho neu can, hoac .close() khi xong).
 """

```

N, K = A_np.shape
K2, M = B_np.shape
assert K == K2, f"K khong khop: A co K={K}, B co K={K2}"
assert N <= 64, f"N={N} vuot gioi han phan cung MAX_N=64"
assert K <= 768, f"K={K} vuot gioi han phan cung MAX_K=768"

if A_buf is None:
    A_buf = allocate(shape=(K, 64), dtype=np.int8) # (K,N) - da transpose, xem
if B_buf is None:
    B_buf = allocate(shape=(K, M), dtype=np.int8)
if C_buf is None:
    C_buf = allocate(shape=(N, M), dtype=np.int32)

if update_A:
    # A_col[k][n] = A_np[n][k] – khop dung layout AXI4_A_READER doc (1 burst =
    # cung k). Day la buoc de quen nhât khi viet lai code moi cho kien truc nay
    A_buf[:, :N] = np.ascontiguousarray(A_np.T) # chỉ N cot dau la du lieu tha
    A_buf.flush()
B_buf[:] = B_np
B_buf.flush()

rm = accel_ip.register_map
rm.A_addr = A_buf.physical_address
rm.B_addr = B_buf.physical_address
rm.C_addr = C_buf.physical_address
rm.N = N
rm.K = K
rm.M = M
rm.update_A = update_A

rm.CTRL.AP_START = 1
while rm.CTRL.AP_DONE == 0:
    pass

C_buf.invalidate()
C_np = np.array(C_buf)
return C_np, C_buf

```

Phần 2 — Kiểm tra nhanh (sanity check) trước khi tin tưởng kết quả DistilBERT

Chạy 1 phép nhân ma trận ngẫu nhiên nhỏ, so khớp NumPy — xác nhận `call_fpga()` đúng trước khi dùng cho toàn bộ phần DistilBERT phía dưới. Nếu bước này FAIL, đừng chạy

tiếp các phần sau.

```
In [3]: rng = np.random.default_rng(42)
N_test, K_test, M_test = 64, 768, 768
A_check = rng.integers(-8, 8, size=(N_test, K_test), dtype=np.int8)
B_check = rng.integers(-8, 8, size=(K_test, M_test), dtype=np.int8)

C_fpga, _ = call_fpga(A_check, B_check, accel_ip, update_A=1)
C_golden = A_check.astype(np.int32) @ B_check.astype(np.int32)

print("Sanity check khớp NumPy:", np.array_equal(C_fpga, C_golden))
assert np.array_equal(C_fpga, C_golden), "FAIL - dừng lại, không chạy tiếp phần Dis
```

Sanity check khớp NumPy: True

```
In [4]: import time # can cho time.perf_counter() o cell benchmark FFN ben duoi

# Case FFN (khop baseline cu 66.999 ms, xem 01_design/docs/task.md muc "D2"):
# N=64 (so token), K=768 (hidden_size DistilBERT), M=3072 (FFN up-projection)
N_ffn, K_ffn, M_ffn = 64, 768, 3072

rng_ffn = np.random.default_rng(123) # seed khac sanity check o tren, tranh trung
A_ffn = rng_ffn.integers(-8, 8, size=(N_ffn, K_ffn), dtype=np.int8)
B_ffn = rng_ffn.integers(-8, 8, size=(K_ffn, M_ffn), dtype=np.int8)
C_golden_ffn = A_ffn.astype(np.int32) @ B_ffn.astype(np.int32)
```

```
In [5]: # Cấp phát + ghi dữ liệu 1 lần (KHÔNG tính vào thời gian đo)
A_buf_ffn = allocate(shape=(K_ffn, N_ffn), dtype=np.int8)
B_buf_ffn = allocate(shape=(K_ffn, M_ffn), dtype=np.int8)
C_buf_ffn = allocate(shape=(N_ffn, M_ffn), dtype=np.int32)

A_buf_ffn[:] = np.ascontiguousarray(A_ffn.T)
B_buf_ffn[:] = B_ffn
A_buf_ffn.flush()
B_buf_ffn.flush()

rm = accel_ip.register_map
rm.A_addr = A_buf_ffn.physical_address
rm.B_addr = B_buf_ffn.physical_address
rm.C_addr = C_buf_ffn.physical_address
rm.N = N_ffn
rm.K = K_ffn
rm.M = M_ffn
rm.update_A = 1

# CHỈ đo đúng đoạn compute (AP_START -> AP_DONE), khớp đúng cách baseline cũ đã đo
t0 = time.perf_counter()
rm.CTRL.AP_START = 1
while rm.CTRL.AP_DONE == 0:
    pass
t_compute = time.perf_counter() - t0

C_buf_ffn.invalidate()
print(f"FFN compute-only: {t_compute*1000:.3f} ms")
print(f"Khớp NumPy: {np.array_equal(np.array(C_buf_ffn), C_golden_ffn)}")
```

```
print(f"So với baseline cũ 66.999 ms: "
      f"{'nhANH hơn' if t_compute*1000 < 66.999 else 'CHẬM HƠN'} "
      f"{abs(66.999 - t_compute*1000)/66.999*100:.1f}%")
```

FFN compute-only: 16.200 ms

Khớp NumPy: True

So với baseline cũ 66.999 ms: nhanh hơn 75.8%

```
In [6]: import time

t0 = time.perf_counter()
C_np_check = A_ffn.astype(np.int32) @ B_ffn.astype(np.int32)
t_numpy_ffn = time.perf_counter() - t0

import torch
A_t = torch.from_numpy(A_ffn.astype(np.int32))
B_t = torch.from_numpy(B_ffn.astype(np.int32))
t0 = time.perf_counter()
C_t_check = A_t @ B_t
t_torch_ffn = time.perf_counter() - t0

print(f"NumPy: {t_numpy_ffn*1000:.3f} ms")
print(f"PyTorch: {t_torch_ffn*1000:.3f} ms")
print(f"FPGA compute-only: {t_compute*1000:.3f} ms")
print(f"\nSpeedup vs NumPy: {t_numpy_ffn/t_compute:.2f}x (bài báo: 214.24x)")
print(f"Speedup vs PyTorch: {t_torch_ffn/t_compute:.2f}x (bài báo: 7.01x)")
```

NumPy: 20920.722 ms

PyTorch: 329.406 ms

FPGA compute-only: 16.200 ms

\nSpeedup vs NumPy: 1291.41x (bài báo: 214.24x)

Speedup vs PyTorch: 20.33x (bài báo: 7.01x)

```
In [7]: # ---- Case Attention (64/768/768) - so sánh với FFN vừa đo ----
N_attn, K_attn, M_attn = 64, 768, 768

A_attn = rng.integers(-8, 8, size=(N_attn, K_attn), dtype=np.int8)
B_attn = rng.integers(-8, 8, size=(K_attn, M_attn), dtype=np.int8)
C_golden_attn = A_attn.astype(np.int32) @ B_attn.astype(np.int32)

A_buf_attn = allocate(shape=(K_attn, N_attn), dtype=np.int8)
B_buf_attn = allocate(shape=(K_attn, M_attn), dtype=np.int8)
C_buf_attn = allocate(shape=(N_attn, M_attn), dtype=np.int32)

A_buf_attn[:] = np.ascontiguousarray(A_attn.T)
B_buf_attn[:] = B_attn
A_buf_attn.flush()
B_buf_attn.flush()

rm = accel_ip.register_map
rm.A_addr = A_buf_attn.physical_address
rm.B_addr = B_buf_attn.physical_address
rm.C_addr = C_buf_attn.physical_address
rm.N = N_attn
rm.K = K_attn
rm.M = M_attn
rm.update_A = 1
```

```

t0 = time.perf_counter()
rm.CTRL.AP_START = 1
while rm.CTRL.AP_DONE == 0:
    pass
t_compute_attn = time.perf_counter() - t0

C_buf_attn.invalidate()
print(f"Attention compute-only: {t_compute_attn*1000:.3f} ms")
print(f"Khớp NumPy: {np.array_equal(np.array(C_buf_attn), C_golden_attn)}")
print(f"So với baseline cũ 17.332 ms: "
      f"{'nhANH hơn' if t_compute_attn*1000 < 17.332 else 'CHẬM HƠN'} "
      f"{{abs(17.332 - t_compute_attn*1000)/17.332*100:.1f}}%")

# So tỷ lệ FFN/Attention - nếu gần tỷ lệ số giao dịch B (9216/2304=4x) hơn là tỷ lệ
# trong trường hợp này vì N,K có định, chỉ M đổi) - cần nhìn cả 2 con số cùng lúc,
print(f"\nTỷ lệ thời gian FFN/Attention: {t_compute*1000/(t_compute_attn*1000):.2f}
      f"(so_giao_dich_B FFN/Attention = {9216/2304:.2f}x, ty_le_FLOPs = {M_ffn/M_at

```

Attention compute-only: 4.666 ms

Khớp NumPy: True

So với baseline cũ 17.332 ms: nhanh hơn 73.1%

\nTỷ lệ thời gian FFN/Attention: 3.47x (so_giao_dich_B FFN/Attention = 4.00x, ty_le_FLOPs = 4.00x)

```

In [8]: # ---- Case N=32 (nửa N so với FFN gốc N=64), cùng K/M - cô lập ảnh hưởng của N ---
N_half = 32

A_half = rng.integers(-8, 8, size=(N_half, K_ffn), dtype=np.int8)
C_golden_half = A_half.astype(np.int32) @ B_ffn.astype(np.int32)

A_buf_half = allocate(shape=(K_ffn, 64), dtype=np.int8) # LUÔN 64 cột (fix giống
C_buf_half = allocate(shape=(N_half, M_ffn), dtype=np.int32)

A_buf_half[:, :N_half] = np.ascontiguousarray(A_half.T) # chỉ ghi 32 cột đầu là d
A_buf_half.flush()
# B_buf_ffn dùng lại nguyên (đã ghi/flush từ trước, cùng B)

rm = accel_ip.register_map
rm.A_addr = A_buf_half.physical_address
rm.B_addr = B_buf_ffn.physical_address
rm.C_addr = C_buf_half.physical_address
rm.N = N_half
rm.K = K_ffn
rm.M = M_ffn
rm.update_A = 1

t0 = time.perf_counter()
rm.CTRL.AP_START = 1
while rm.CTRL.AP_DONE == 0:
    pass
t_compute_half = time.perf_counter() - t0

C_buf_half.invalidate()
print(f"N=32 (K=768,M=3072) compute-only: {t_compute_half*1000:.3f} ms")
print(f"Khớp NumPy: {np.array_equal(np.array(C_buf_half), C_golden_half)}")

```

```
print(f"\nSo với N=64 (t_compute={t_compute*1000:.3f} ms): "
      f"tỷ lệ = {t_compute/t_compute_half:.2f}x")
print(f"Nếu chỉ compute+C phụ thuộc N: kỳ vọng tỷ lệ ~2.0x")
print(f"Nếu có 'sàn' cố định do B/A không đổi theo N: tỷ lệ sẽ NHỎ HƠN 2.0x rõ rệt")
```

N=32 (K=768,M=3072) compute-only: 13.832 ms

Khớp NumPy: True

\nSo với N=64 (t_compute=16.200 ms): tỷ lệ = 1.17x

Nếu chỉ compute+C phụ thuộc N: kỳ vọng tỷ lệ ~2.0x

Nếu có 'sàn' cố định do B/A không đổi theo N: tỷ lệ sẽ NHỎ HƠN 2.0x rõ rệt

Phần 3 — `FPGAQuantizedLinear`: thay 1 lớp `nn.Linear` bằng bản chạy FPGA

Lượng tử hoá động (per-call): tính scale từ giá trị lớn nhất tuyệt đối của activation/weight hiện tại (kiểu "demo" đơn giản, giống cách bài báo tham khảo làm — không cần calibration set riêng). Dùng cho việc thay từng lớp riêng lẻ (không chia sẻ A giữa nhiều lớp) — Phần 4 sẽ có bản tối ưu hơn cho đúng 3 lớp Q/K/V cùng chung 1 input.

```
In [9]: import torch
import torch.nn as nn

class FPGAQuantizedLinear(nn.Module):
    """Thay 1 nn.Linear bằng bản chạy FPGA. K = in_features phải <= 768 (xem giới hạn
    cung ở đầu notebook) – kiểm tra ngay lúc khởi tạo, báo lỗi rõ ràng thay vì de F

    def __init__(self, linear: nn.Linear, accel_ip):
        super().__init__()
        assert linear.in_features <= 768, (
            f"in_features={linear.in_features} vượt giới hạn K<=768 của phần cứng –
            f"không offload được lớp này (vd FFN down-projection), giữ nguyên CPU."
        )
        self.in_features = linear.in_features
        self.out_features = linear.out_features
        self.accel_ip = accel_ip

        W = linear.weight.detach().cpu().numpy() # shape (out_features, in_features)
        self.bias = (linear.bias.detach().cpu().numpy().astype(np.float32)
                     if linear.bias is not None else None)

        w_scale = np.abs(W).max() / 127.0
        self.w_scale = w_scale if w_scale > 0 else 1e-8
        W_int8 = np.round(W / self.w_scale).astype(np.int8)
        # B_np can shape (K, M) = (in_features, out_features) -> transpose W, KHÔNG
        # thêm nữa ở call_fpga (B không cần transpose, chỉ A mới cần).
        self.B_int8 = np.ascontiguousarray(W_int8.T)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        orig_shape = x.shape
        x2d = x.reshape(-1, self.in_features).detach().cpu().numpy()
        N = x2d.shape[0]
```

```

assert N <= 64, f"N={N} (so token/hang) vuot gioi han phan cung MAX_N=64"

x_scale = np.abs(x2d).max() / 127.0
x_scale = x_scale if x_scale > 0 else 1e-8
X_int8 = np.round(x2d / x_scale).astype(np.int8)

C_int32, C_buf = call_fpga(X_int8, self.B_int8, self.accel_ip, update_A=1)
C_buf.close()

out = C_int32.astype(np.float32) * (x_scale * self.w_scale)
if self.bias is not None:
    out = out + self.bias
return torch.from_numpy(out).reshape(*orig_shape[:-1], self.out_features)

```

Kiểm tra nhanh FPGAQuantizedLinear trên 1 lớp thật

So sánh output 1 lớp `q_lin` thật của DistilBERT (chưa gắn vào model) — CPU vs FPGA, trước khi thay cả 3 lớp Q/K/V trong toàn bộ model.

```

In [10]: from transformers import AutoModel

model_name = "distilbert-base-uncased"
model = AutoModel.from_pretrained(model_name)
model.eval()

sample_input = torch.randn(32, 768) * 0.5 # gia lap 32 token, hidden=768
layer0 = model.transformer.layer[0]
original_q_lin = layer0.attention.q_lin
fpga_q_lin = FPGAQuantizedLinear(original_q_lin, accel_ip)

with torch.no_grad():
    out_cpu = original_q_lin(sample_input)
    out_fpga = fpga_q_lin(sample_input)

rel_error = ((out_cpu - out_fpga).norm() / out_cpu.norm() * 100).item()
print(f"Sai số tương đối (norm) CPU vs FPGA cho 1 lớp q_lin: {rel_error:.3f}%")
print("(Sai số > 0 là bình thường – do lượng tử hoá int8, không phải bug; so ~1-3%)")

```

Loading weights: 0% | 0/100 [00:00<?, ?it/s]

[transformers] DistilBertModel LOAD REPORT from: distilbert-base-uncased

Key	Status	
vocab_layer_norm.weight	UNEXPECTED	
vocab_transform.weight	UNEXPECTED	
vocab_layer_norm.bias	UNEXPECTED	
vocab_projector.bias	UNEXPECTED	
vocab_transform.bias	UNEXPECTED	

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Sai số tương đối (norm) CPU vs FPGA cho 1 lớp `q_lin`: 3.358%

(Sai số > 0 là bình thường – do lượng tử hoá int8, không phải bug; so ~1-3% là hợp lý)

Phần 4 — FPGAQKVAttention: gộp Q/K/V, chia sẻ 1 buffer A (persistent-A)

Q, K, V đều nhận **cùng 1 input** (hidden states của layer) — chỉ khác trọng số. Tận dụng cơ chế `update_A` của phần cứng: nạp A (activation) đúng 1 lần cho Q (`update_A=1`), K và V dùng lại A đã nạp (`update_A=0` , phần cứng đọc lại từ BRAM, không đọc lại DDR) — giảm 2/3 lưu lượng đọc A so với gọi `FPGAQuantizedLinear` 3 lần độc lập.

```
In [11]: class FPGAQKVAttention:
    """Gop Q/K/V, dung chung 1 buffer A (persistent-A qua update_A). Khác FPGAQuant
    day KHONG PHAI nn.Module – dung qua FPGAQKVWrapper (nn.Module moi lop) de gan v
    vi 3 wrapper (q/k/v) can CHIA SE cung 1 instance nay (tinh ca A_buf/C_buf) thay
    tu tao rieng."""

    def __init__(self, q_lin, k_lin, v_lin, accel_ip, max_n=64):
        assert q_lin.in_features <= 768, "K vuot gioi han phan cung, xem markdown d
        self.accel_ip = accel_ip
        self.in_features = q_lin.in_features
        self.out_features = q_lin.out_features
        self.max_n = max_n

        self.biases, self.w_scales, self.B_int8_list = [], [], []
        for lin in (q_lin, k_lin, v_lin):
            W = lin.weight.detach().cpu().numpy()
            bias = (lin.bias.detach().cpu().numpy().astype(np.float32)
                    if lin.bias is not None else None)
            w_scale = np.abs(W).max() / 127.0
            w_scale = w_scale if w_scale > 0 else 1e-8
            W_int8 = np.round(W / w_scale).astype(np.int8)
            self.biases.append(bias)
            self.w_scales.append(w_scale)
            self.B_int8_list.append(np.ascontiguousarray(W_int8.T))

        K, M = self.in_features, self.out_features
        # Cap phat san 1 lan, tai su dung qua nhieu lan goi (tranh alloc/free lien
        # allocate() cua PYNQ khong re, gap nhieu lan trong 1 forward pass se cham)
        self.A_buf = allocate(shape=(K, max_n), dtype=np.int8) # da transpose: (K,
        self.B_buf = allocate(shape=(K, M), dtype=np.int8)
        self.C_buf = allocate(shape=(max_n, M), dtype=np.int32)
        self._cache = None # cache 3 output (Q,K,V) cho 1 lan forward, dung boi Wr

    def _run_one(self, N, B_int8, update_A):
        self.B_buf[:, :] = B_int8
        self.B_buf.flush()
        if update_A:
            self.A_buf.flush() # A da duoc ghi truoc do trong compute_qkv()

        rm = self.accel_ip.register_map
        rm.A_addr = self.A_buf.physical_address
        rm.B_addr = self.B_buf.physical_address
        rm.C_addr = self.C_buf.physical_address
        rm.N = N
```

```

rm.K = self.in_features
rm.M = self.out_features
rm.update_A = 1 if update_A else 0

rm.CTRL.AP_START = 1
while rm.CTRL.AP_DONE == 0:
    pass

self.C_buf.invalidate()
return np.array(self.C_buf[:N, :])

def compute_qkv(self, x: torch.Tensor):
    orig_shape = x.shape
    x2d = x.reshape(-1, self.in_features).detach().cpu().numpy()
    N, K = x2d.shape
    assert N <= self.max_n, f"N={N} vuot max_n={self.max_n} da cap phat cho buf

    x_scale = np.abs(x2d).max() / 127.0
    x_scale = x_scale if x_scale > 0 else 1e-8
    X_int8 = np.round(x2d / x_scale).astype(np.int8)

    # Ghi A 1 LAN DUY NHAT (transpose truoc, khop layout AXI4_A_READER) - Q/K/V
    # Lai dung buffer nay, chi K/V bao update_A=0 de phan cung khong doc lai DD
    self.A_buf[:, :N] = np.ascontiguousarray(X_int8.T)

    outputs = []
    for i in range(3): # 0=Q, 1=K, 2=V
        C_int32 = self._run_one(N, self.B_int8_list[i], update_A=(i == 0))
        out = C_int32.astype(np.float32) * (x_scale * self.w_scales[i])
        if self.biases[i] is not None:
            out = out + self.biases[i]
        outputs.append(torch.from_numpy(out).reshape(*orig_shape[:-1], self.out
    return outputs

class FPGAQKVWrapper(nn.Module):
    """nn.Module riêng cho tung lop (q/k/v), tro chung ve 1 FPGAQKVAttention. Lan g
    tien (index=0, tuc q_lin) trong 1 forward pass se tinh ca 3 va cache lai - k_li
    sau chi lay tu cache, khong tinh lai."""

    def __init__(self, shared: FPGAQKVAttention, index: int):
        super().__init__()
        self.shared = shared
        self.index = index

    def forward(self, x):
        if self.index == 0:
            self.shared._cache = self.shared.compute_qkv(x)
        return self.shared._cache[self.index]

```

Phần 5 — Gắn vào toàn bộ DistilBERT, so sánh CPU vs FPGA (hidden states)

Thay `q_lin / k_lin / v_lin` của cả 6 lớp transformer bằng `FPGAQKVWrapper` chia sẻ chung 1 `FPGAQKVAttention` /layer. So sánh `last_hidden_state` giữa bản CPU gốc và bản FPGA — sai số tương đối (norm) + cosine similarity, không chỉ đo tốc độ.

```
In [12]: import copy
import time

model_fpga = copy.deepcopy(model)
for layer in model_fpga.transformer.layer:
    shared = FPGAQKVAttention(layer.attention.q_lin, layer.attention.k_lin,
                              layer.attention.v_lin, accel_ip, max_n=64)
    layer.attention.q_lin = FPGAQKVWrapper(shared, 0)
    layer.attention.k_lin = FPGAQKVWrapper(shared, 1)
    layer.attention.v_lin = FPGAQKVWrapper(shared, 2)
print(f"Đã thay xong {len(model_fpga.transformer.layer)} lớp Q/K/V bằng bản FPGA (p

from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(model_name)

def run_hidden_state_test(sentence, max_length=64):
    """So sanh last_hidden_state CPU vs FPGA cho 1 cau, luon pad/truncate ve max_le
    (mac dinh 64 – khop dung gioi han phan cung MAX_N)."""
    inputs = tokenizer(sentence, return_tensors="pt", padding="max_length",
                       max_length=max_length, truncation=True)
    with torch.no_grad():
        t0 = time.time()
        out_cpu = model(**inputs)
        t_cpu = time.time() - t0

        t0 = time.time()
        out_fpga = model_fpga(**inputs)
        t_fpga = time.time() - t0

    hc, hf = out_cpu.last_hidden_state, out_fpga.last_hidden_state
    rel_err = ((hc - hf).norm() / hc.norm() * 100).item()
    cos = torch.nn.functional.cosine_similarity(hc.flatten(), hf.flatten(), dim=0).
    print(f"[{max_length} token] CPU={t_cpu*1000:.1f}ms  FPGA={t_fpga*1000:.1f}ms
          f"speedup={t_cpu/t_fpga:.2f}x  SaiSố={rel_err:.2f}%  Cosine={cos:.4f}")
    return t_cpu, t_fpga, rel_err, cos

print("\n-- Câu ngắn --")
run_hidden_state_test("The quick brown fox jumps over the lazy dog.")
print("\n-- Câu dài (gần chạm giới hạn 64 token) --")
run_hidden_state_test(
    "The quick brown fox jumps over the lazy dog while the sun sets slowly behind t
    "distant mountains, casting long shadows across the quiet valley below where a
    "river flows gently through the fields."
)
```

Đã thay xong 6 lớp Q/K/V bằng bản FPGA (persistent-A).

-- Câu ngắn --

[64 token] CPU=1102.4ms FPGA=606.8ms speedup=1.82x SaiSố=12.39% Cosine=0.9923

-- Câu dài (gần chạm giới hạn 64 token) --

[64 token] CPU=632.1ms FPGA=607.0ms speedup=1.04x SaiSố=10.00% Cosine=0.9950

Out[12]: (0.6321361064910889, 0.60701584815979, 10.003976821899414, 0.9949963092803955)

Phần 6 — Đánh giá end-to-end: phân loại cảm xúc (sentiment classification)

Dùng `distilbert-base-uncased-finetuned-sst-2-english` — so nhãn dự đoán + độ tin cậy giữa CPU và bản offload FPGA trên nhiều câu, không chỉ 1-2 câu đơn lẻ.

```
In [13]: from transformers import AutoModelForSequenceClassification
import torch.nn.functional as F

clf_model_name = "distilbert-base-uncased-finetuned-sst-2-english"
clf_tokenizer = AutoTokenizer.from_pretrained(clf_model_name)
clf_model = AutoModelForSequenceClassification.from_pretrained(clf_model_name)
clf_model.eval()

clf_model_fpga = copy.deepcopy(clf_model)
for layer in clf_model_fpga.distilbert.transformer.layer:
    shared = FPGAQKVAttention(layer.attention.q_lin, layer.attention.k_lin,
                              layer.attention.v_lin, accel_ip, max_n=64)
    layer.attention.q_lin = FPGAQKVWrapper(shared, 0)
    layer.attention.k_lin = FPGAQKVWrapper(shared, 1)
    layer.attention.v_lin = FPGAQKVWrapper(shared, 2)

test_sentences = [
    "This movie was absolutely fantastic, I loved every minute of it!",
    "The service was terrible and I will never come back again.",
    "It was an okay experience, nothing special but not bad either.",
    "Best purchase I have made all year, highly recommend to everyone.",
    "Complete waste of money, do not buy this product.",
    "The food was delicious and the staff were very friendly.",
    "I am extremely disappointed with the quality of this item.",
    "A truly wonderful performance, the actors were brilliant.",
    "This is the worst customer service I have ever experienced.",
    "Pretty good overall, met my expectations for the price.",
    "The plot was boring and the pacing felt way too slow.",
    "Amazing product, works exactly as described, five stars.",
    "I regret buying this, it broke after just one use.",
    "The hotel room was clean, comfortable, and had a great view.",
    "Terrible experience from start to finish, avoid at all costs.",
    "Solid effort, though there is definitely room for improvement.",
]

agree = 0
conf_diffs = []
for sentence in test_sentences:
```

```

inputs_s = clf_tokenizer(sentence, return_tensors="pt")
with torch.no_grad():
    out_cpu_s = clf_model(**inputs_s)
    out_fpga_s = clf_model_fpga(**inputs_s)
prob_cpu = F.softmax(out_cpu_s.logits, dim=-1)[0]
prob_fpga = F.softmax(out_fpga_s.logits, dim=-1)[0]
label_cpu = clf_model.config.id2label[prob_cpu.argmax().item()]
label_fpga = clf_model.config.id2label[prob_fpga.argmax().item()]
match = (label_cpu == label_fpga)
agree += int(match)
conf_diff = abs(prob_cpu.max().item() - prob_fpga.max().item()) * 100
conf_diffs.append(conf_diff)
tag = "OK " if match else "SAI"
print(f"{tag} CPU={label_cpu}({prob_cpu.max().item()*100:.1f}%) "
      f"FPGA={label_fpga}({prob_fpga.max().item()*100:.1f}%) "
      f"lệch={conf_diff:.2f}% | {sentence[:45]}")

print(f"\n=== Tổng kết trên {len(test_sentences)} câu ===")
print(f"Tỷ lệ khớp nhãn: {agree}/{len(test_sentences)} = {agree/len(test_sentences)}")
print(f"Độ lệch confidence trung bình: {sum(conf_diffs)/len(conf_diffs):.3f} điểm %")
print(f"Độ lệch confidence lớn nhất: {max(conf_diffs):.3f} điểm %")

```

Loading weights: 0% | 0/104 [00:00<?, ?it/s]

OK CPU=POSITIVE(100.0%) FPGA=POSITIVE(100.0%) lệch=0.00% | This movie was absolutely fantastic, I loved

OK CPU=NEGATIVE(99.7%) FPGA=NEGATIVE(99.7%) lệch=0.03% | The service was terrible and I will never com

OK CPU=POSITIVE(98.6%) FPGA=POSITIVE(98.4%) lệch=0.18% | It was an okay experience, nothing special bu

OK CPU=POSITIVE(100.0%) FPGA=POSITIVE(100.0%) lệch=0.00% | Best purchase I have made all year, highly re

OK CPU=NEGATIVE(100.0%) FPGA=NEGATIVE(100.0%) lệch=0.00% | Complete waste of money, do not buy this prod

OK CPU=POSITIVE(100.0%) FPGA=POSITIVE(100.0%) lệch=0.00% | The food was delicious and the staff were ver

OK CPU=NEGATIVE(100.0%) FPGA=NEGATIVE(100.0%) lệch=0.00% | I am extremely disappointed with the quality

OK CPU=POSITIVE(100.0%) FPGA=POSITIVE(100.0%) lệch=0.00% | A truly wonderful performance, the actors wer

OK CPU=NEGATIVE(100.0%) FPGA=NEGATIVE(100.0%) lệch=0.00% | This is the worst customer service I have eve

OK CPU=POSITIVE(100.0%) FPGA=POSITIVE(100.0%) lệch=0.00% | Pretty good overall, met my expectations for

OK CPU=NEGATIVE(100.0%) FPGA=NEGATIVE(100.0%) lệch=0.00% | The plot was boring and the pacing felt way t

OK CPU=POSITIVE(100.0%) FPGA=POSITIVE(100.0%) lệch=0.00% | Amazing product, works exactly as described,

OK CPU=NEGATIVE(99.9%) FPGA=NEGATIVE(100.0%) lệch=0.00% | I regret buying this, it broke after just one

OK CPU=POSITIVE(100.0%) FPGA=POSITIVE(100.0%) lệch=0.00% | The hotel room was clean, comfortable, and ha

OK CPU=NEGATIVE(99.9%) FPGA=NEGATIVE(99.9%) lệch=0.00% | Terrible experience from start to finish, avo

OK CPU=POSITIVE(99.6%) FPGA=POSITIVE(99.6%) lệch=0.03% | Solid effort, though there is definitely room

=== Tổng kết trên 16 câu ===

Tỷ lệ khớp nhãn: 16/16 = 100.0%

Độ lệch confidence trung bình: 0.016 điểm %

Độ lệch confidence lớn nhất: 0.180 điểm %

Phần 7 — Hiệu năng: FLOPs, throughput, so với bài báo tham khảo

Tính FLOPs riêng phần Q/K/V (phần đã offload) so với tổng FLOPs cả model — để hiểu vì sao speedup tổng thể (end-to-end) luôn thấp hơn nhiều so với speedup riêng phép MatMul (đúng kết luận bài báo tham khảo: "QKV projection alone does not account for a large portion of total compute").

```
In [14]: cfg = model.config
seq_len = 64
n_layers = cfg.n_layers
hidden = cfg.dim
ffn_dim = cfg.hidden_dim
n_heads = cfg.n_heads
d_k = hidden // n_heads
```

```

flops_qkv = 3 * 2 * seq_len * hidden * hidden # phần DA offload Len FPGA
flops_qkt = 2 * n_heads * seq_len * seq_len * d_k
flops_av = 2 * n_heads * seq_len * seq_len * d_k
flops_oproj = 2 * seq_len * hidden * hidden
flops_ffn = 2 * seq_len * hidden * ffn_dim + 2 * seq_len * ffn_dim * hidden # Ch

flops_per_layer = flops_qkv + flops_qkt + flops_av + flops_oproj + flops_ffn
total_flops = flops_per_layer * n_layers
qkv_share = flops_qkv * n_layers / total_flops * 100

print(f"FLOPs Q/K/V (đã offload FPGA)/lớp: {flops_qkv/1e6:.2f} MFLOPs")
print(f"FLOPs mỗi lớp (toàn bộ): {flops_per_layer/1e6:.2f} MFLOPs")
print(f"Tổng FLOPs ({n_layers} lớp): {total_flops/1e9:.3f} GFLOPs")
print(f"Tỷ trọng Q/K/V trong tổng FLOPs: {qkv_share:.1f}% "
      f"(phần còn lại – attention score, FFN – vẫn chạy CPU)")

sentence = "The quick brown fox jumps over the lazy dog."
inputs = tokenizer(sentence, return_tensors="pt", padding="max_length",
                  max_length=seq_len, truncation=True)
with torch.no_grad():
    t0 = time.time(); out_cpu_full = model(**inputs); t_cpu_full = time.time() - t0
    t0 = time.time(); out_fpga_full = model_fpga(**inputs); t_fpga_full = time.time()

print(f"\nCPU-only          : {t_cpu_full*1000:.2f} ms -> "
      f"{total_flops/t_cpu_full/1e9:.3f} GFLOPs/s")
print(f"FPGA (Q/K/V offload): {t_fpga_full*1000:.2f} ms -> "
      f"{total_flops/t_fpga_full/1e9:.3f} GFLOPs/s")
print(f"Speedup end-to-end: {t_cpu_full/t_fpga_full:.2f}x "
      f"(kỳ vọng khiêm tốn – đúng như phân tích tỷ trọng FLOPs ở trên)")

```

FLOPs Q/K/V (đã offload FPGA)/lớp: 226.49 MFLOPs

FLOPs mỗi lớp (toàn bộ): 918.55 MFLOPs

Tổng FLOPs (6 lớp): 5.511 GFLOPs

Tỷ trọng Q/K/V trong tổng FLOPs: 24.7% (phần còn lại – attention score, FFN – vẫn chạy CPU)

CPU-only : 617.95 ms -> 8.919 GFLOPs/s

FPGA (Q/K/V offload): 588.87 ms -> 9.359 GFLOPs/s

Speedup end-to-end: 1.05x (kỳ vọng khiêm tốn – đúng như phân tích tỷ trọng FLOPs ở trên)

Phần 8 — Công suất & năng lượng (tuỳ chọn, cần cảm biến hwmon2 trên board)

Đo công suất hệ thống qua `/sys/class/hwmon/hwmon2/power1_input` (đường dẫn cảm biến điện năng trên KV260 — đã dùng và xác nhận đúng trong các notebook trước của dự án). Nếu board không có đường dẫn này (khác model/khác cấu hình PMBus), bỏ qua phần này, không ảnh hưởng các phần trước.

In [15]: `import threading`

```

def read_power_watts():
    with open("/sys/class/hwmon/hwmon2/power1_input") as f:
        return int(f.read().strip()) / 1_000_000

def measure_power_during(func, *args, sample_interval=0.05, **kwargs):
    samples = []
    stop_flag = threading.Event()

    def sampler():
        while not stop_flag.is_set():
            samples.append(read_power_watts())
            time.sleep(sample_interval)

    t = threading.Thread(target=sampler)
    t.start()
    t0 = time.time()
    result = func(*args, **kwargs)
    elapsed = time.time() - t0
    stop_flag.set()
    t.join()

    avg_power = sum(samples) / len(samples) if samples else read_power_watts()
    energy_j = avg_power * elapsed
    return result, avg_power, energy_j, elapsed, len(samples)

idle_samples = [read_power_watts() for _ in range(20)]
time.sleep(0.05)
idle_power = sum(idle_samples) / len(idle_samples)
print(f"Công suất idle (baseline): {idle_power:.3f} W")

sentence = "The quick brown fox jumps over the lazy dog."
inputs_pw = tokenizer(sentence, return_tensors="pt", padding="max_length",
                      max_length=64, truncation=True)

def run_cpu_once():
    with torch.no_grad():
        return model(**inputs_pw)

def run_fpga_once():
    with torch.no_grad():
        return model_fpga(**inputs_pw)

_, power_cpu, energy_cpu, t_cpu_pw, _ = measure_power_during(run_cpu_once)
print(f"CPU : công suất TB={power_cpu:.3f}W  thời gian={t_cpu_pw*1000:.1f}ms  "
      f"năng lượng={energy_cpu*1000:.2f}mJ")

_, power_fpga, energy_fpga, t_fpga_pw, _ = measure_power_during(run_fpga_once)
print(f"FPGA : công suất TB={power_fpga:.3f}W  thời gian={t_fpga_pw*1000:.1f}ms  "
      f"năng lượng={energy_fpga*1000:.2f}mJ")

```

```
if energy_fpga > 0:
    print(f"\nHiệu suất năng lượng FPGA vs CPU: {energy_cpu/energy_fpga:.2f}x tốt hơn")
```

Công suất idle (baseline): 3.448 W

CPU : công suất TB=4.230W thời gian=620.4ms năng lượng=2624.41mJ

FPGA : công suất TB=4.083W thời gian=603.0ms năng lượng=2461.87mJ

Hiệu suất năng lượng FPGA vs CPU: 1.07x tốt hơn

```
In [16]: import statistics

sentence = "The quick brown fox jumps over the lazy dog."
inputs = tokenizer(sentence, return_tensors="pt", padding="max_length", max_length=

n_reps = 8
t_cpu_list, t_fpga_list = [], []

for i in range(n_reps):
    with torch.no_grad():
        t0 = time.time()
        model(**inputs)
        t_cpu_list.append(time.time() - t0)

        t0 = time.time()
        model_fpga(**inputs)
        t_fpga_list.append(time.time() - t0)
    print(f"Lần {i+1}: CPU={t_cpu_list[-1]*1000:.1f}ms  FPGA={t_fpga_list[-1]*1000:.1f}ms")

t_cpu_med = statistics.median(t_cpu_list)
t_fpga_med = statistics.median(t_fpga_list)

print(f"\n=== Kết quả sau {n_reps} lần đo ===")
print(f"CPU : median={t_cpu_med*1000:.1f}ms (min={min(t_cpu_list)*1000:.1f}, max={max(t_cpu_list)*1000:.1f})")
print(f"FPGA : median={t_fpga_med*1000:.1f}ms (min={min(t_fpga_list)*1000:.1f}, max={max(t_fpga_list)*1000:.1f})")
print(f"Speedup (median, đáng tin hơn 1 lần đo): {t_cpu_med/t_fpga_med:.2f}x")
```

Lần 1: CPU=620.2ms FPGA=587.1ms

Lần 2: CPU=617.1ms FPGA=587.6ms

Lần 3: CPU=611.2ms FPGA=589.5ms

Lần 4: CPU=611.3ms FPGA=586.2ms

Lần 5: CPU=619.2ms FPGA=591.9ms

Lần 6: CPU=612.9ms FPGA=586.9ms

Lần 7: CPU=611.2ms FPGA=588.0ms

Lần 8: CPU=610.2ms FPGA=589.1ms

=== Kết quả sau 8 lần đo ===

CPU : median=612.1ms (min=610.2, max=620.2)

FPGA : median=587.8ms (min=586.2, max=591.9)

Speedup (median, đáng tin hơn 1 lần đo): 1.04x

Tổng kết

- Kiểm tra đúng thứ tự: **Phần 2 (sanity check) phải PASS trước** — nếu FAIL, mọi con số ở Phần 5-8 đều không đáng tin (lỗi transpose/register sẽ cho kết quả "chạy được, có số, nhưng sai").

- Phạm vi offload hiện tại: **chỉ Q/K/V projection** (giới hạn $K \leq 768$ của phần cứng) — attention score/softmax, FFN vẫn chạy CPU. Đây là lý do speedup end-to-end luôn thấp hơn nhiều so với speedup đo riêng phép MatMul (xem phân tích tỷ trọng FLOPs ở Phần 7).
- Muốn mở rộng offload thêm FFN up-projection (`Linear(768, 3072)` , $K=768$, offload được) — có thể thêm 1 `FPGAQuantizedLinear` riêng cho `layer.ffn.lin1` của mỗi lớp, tương tự Phần 3. FFN down-projection (`Linear(3072, 768)`) không offload được trực tiếp — cần tiling K thành nhiều lần gọi 768 phần tử rồi cộng dồn kết quả ở phần mềm, chưa làm trong notebook này.